

K⁻p 散乱イベントディスプレイ（ビーム軌跡つき）

kpsc_event_display_rk.py 使い方手引き

2026 年 10 月 1 日

1 概要

kpsc_event_display_rk.py は、K⁻p 散乱解析の 1 イベントを、TPC のトラックとビームの軌跡をあわせて表示する Python スクリプトです。DstTPCHelixTracking（または -Geant4）の出力と、DstKpScattering（または -Geant4）の出力の 2 つを読み、次の 4 面を 1 枚にまとめます。

- 3D 表示
- 上面図 ($z-x$)
- 側面図 ($z-y$)
- 数値の欄（陽子と K の運動量、弾性散乱からの期待値、mmass など）

実データと Geant4 (G4) の両方に対応しています。1 イベントを PNG に描くことも、選択条件を満たす N イベントを 1 つの PDF にまとめることもできます。

- I/O は uproot を使います (ROOT/PyROOT は使いません)。
- helix の計算 (θ の範囲を含む)、座標変換、TPC の枠の描画は、同じ macro/ 内の check_tpc_helix_track_3d.py から import して再利用しています。helix の描き方はそのマクロと同じです。

2 動作環境

- Python 3, uproot, awkward, matplotlib, numpy
- スクリプトの場所: myanalysis/macro/kpsc_event_display_rk.py。check_tpc_helix_track_3d.py に依存するので、この 2 つは同じディレクトリに置きます。
- 出力はファイルのみです (matplotlib の Agg バックエンドを使うので、画面は開きません)。X11 転送は不要です。

3 入力ファイル

	実データ	G4
--helix (ツリー tpc)	DstTPCHelixTracking の出力 例: /group/had/sks/Users/sryuta/ JPARC2025E72/run03774_TPCHelix. root	DstTPCHelixTrackingGeant4 の出力 (反 応の真値ブランチつき) 例: scratch_root/g4_kp_mom735/ helix_B1T_truth_n100k.root
--kpsc (ツリー kpsc)	DstKpScattering の出力 例: root/run03774/run03774_ KpScattering_lh2off.root	DstKpScatteringGeant4 の出力 例: scratch_root/g4_kp_mom735/kpsc_ B1T_hon_off_s0_n100k.root
--g4	付けない	付ける

2つのファイルは同じイベントを同じ順で含んでいる必要があります (--kpsc は --helix を入力に作ったもの)。実データのビームの軌跡には kpsc の xvp, yvp, zvp (HSTrack の RK で運んだビームの経路) を使います。G4 では、kpsc の beam_used_*・vtx_*_true・p_p_true などと、helix の beam_prm_*・prm_k_* を使います。

4 基本的な使い方

4.1 1 イベントを描く

実データは run と event で指定します。

```
cd /home/had/sryuta/analyzer/JPARC2025E72/myanalysis/macro
D=/group/had/sks/Users/sryuta/JPARC2025E72
python3 kpsc_event_display_rk.py \
  --helix $D/run03774_TPCHelix.root \
  --kpsc $D/root/run03774/run03774_KpScattering_lh2off.root \
  --run 3774 --event 560 --out ev_3774_560.png
```

G4 は event_number が一意でないので、kpsc のエントリ番号で指定します (--event でも描けますが、同じ番号が複数あれば最初のエントリになり、その旨を表示します)。

```
G=/group/had/sks/Users/sryuta/JPARC2025E72/scratch_root/g4_kp_mom735
python3 kpsc_event_display_rk.py --g4 \
  --helix $G/helix_B1T_truth_n100k.root \
  --kpsc $G/kpsc_B1T_hon_off_s0_n100k.root \
  --entry 25 --out ev_g4_25.png
```

出力の形式は --out の拡張子で決まります (.png, .pdf など)。

4.2 選択条件を満たす N イベントを 1 つの PDF にまとめる

```
D=/group/had/sks/Users/sryuta/JPARC2025E72
O=/home/had/sryuta/JPARC2025E72/results/img/run00000/kpsc_lh2eloss/event_display
python3 kpsc_event_display_rk.py \
  --helix $D/run03774_TPCHelix.root \
  --kpsc $D/root/run03774/run03774_KpScattering_lh2off.root \
  --n 12 --out $O/kpsc_event_display_data_run03774.pdf
```

1 ページに 1 イベントです。--skip M を付けると、選ばれたイベントのうち先頭の M 個を飛ばして次から描きます。

4.3 オプション一覧

オプション	意味
--helix FILE	helix tracking の出力 (必須)
--kpsc FILE	KpScattering の出力 (必須)
--g4	G4 のファイルとして扱う
--run R --event E	1 イベントを run と event で指定 (実データ)
--event E	1 イベントを event で指定 (G4 では最初に見つかったエントリ)
--entry I	1 イベントを kpsc のエントリ番号で指定
--n N	選択条件 (§6) を満たすイベントを N 個描いて 1 つの PDF にする
--skip M	--n のとき、選ばれたイベントの先頭 M 個を飛ばす
--label TEXT	図の見出しの文字列 (既定: real data / G4 (honest chain))
--out FILE	出力ファイル (必須)

5 表示内容

描画	内容
灰色の線	すべての TPC helix
赤の線・点	陽子として選ばれたトラック (kpsc の <code>i_p</code>) とそのヒット
青の線・点	K 候補として選ばれたトラック (kpsc の <code>i_kcand</code>) とそのヒット
灰色の破線	<code>is_beam</code> のトラック
黒の星	再構成した頂点 (kpsc の <code> vtx_x</code> , <code> vtx_y</code> , <code> vtx_z</code>)
灰色の円・長方形	LH2 円柱 (半径 40 mm、 $ y \leq 50$ mm、中心 $z = -143$ mm)。上面図では円、側面図では長方形
橙の線 (実データ)	RK で運んだビームの経路 (kpsc の <code>xvp</code> , <code>yvp</code> , <code>zvp</code>)
橙の線 (G4)	運動学に使ったビーム (<code>beam_used_*</code>) を頂点まで延ばした直線
紫の破線 (G4)	<code>beam_prm_*</code> の直線。合成サンプルのビーム側のイベント ($z = -1150$ mm で生成) で、反応に入る K^- とは別の粒子
赤・青の点線 (G4)	真の頂点から伸ばした、真の陽子・K の方向

3D 表示の座標の並びは `check_tpc_helix_track_3d.py` と同じで、matplotlib の $(X, Y, Z) = \text{TPC 局所座標}$ の (z, x, y) です。上面図・側面図の軸は TPC 局所座標をそのまま使っています。

5.1 数値の欄

- `|p_beam|`: 運動学に使ったビームの運動量 (kpsc の `d5_momentum`)
- `mmass - m_K, diff_angle, close_dist`, 頂点の座標
- 陽子と K それぞれについて:
 - `theta_lab`: ビームに対する実験室系の角度
 - `p_meas`: TPC で測った運動量 (kpsc の `p_p`, `p_k`)
 - `p_calc`: `|p_beam|` と `theta_lab` から、 K^-p 弾性散乱の二体運動学 (標的の陽子は静止) で計算した運動量
 - `dk = 1/p_meas - 1/p_calc` $[(\text{GeV}/c)^{-1}]$
- 陽子・K のトラックのヒット数と helix の曲率半径
- G4 では真の運動量 (`p_p_true`, `p_k_true`)

kpsc を LH2 eloss 補正ありで作った場合、陽子の `p_meas` は補正後の値になります。

6 --n の選択条件

次をすべて満たすイベントを、ファイルの先頭から順に選びます。

- `mmass > 0`, `effective_ntTpc == 2`, `close_dist < 5 mm`
- PID (すべて満たす) :
 - `|nsigma_k_sel| < 2`
 - `!(|nsigma_pi_sel| < 3 && nsigma_k_sel < -2)`
 - `nsigma_p_sel > -1.5`
- 頂点が LH2 の中心部: $\sqrt{x^2 + (z + 143)^2} < 40 \text{ mm}$ かつ $|y| < 50 \text{ mm}$
- 弾性散乱らしい方向: `diff_angle < 0.1`, かつ $||\text{delta_phi_scat}| - \pi| < 0.15$

別の条件で選びたいときは、スクリプトの関数 `elastic_mask` を書き換えます。

7 イベントの対応づけ

kpsc のツリーは helix のツリーからエントリ順に作られるので、kpsc のエントリ i と helix のエントリ i は同じイベントです。スクリプトは、両者の `event_number` (実データでは `run_number` も) がそのエントリで一致すればエントリ番号で対応づけます。一致しない場合、実データでは (run, event) の索引で探します。G4 では `event_number` が一意でない (合成した G4 ファイルに同一のジョブブロックが含まれる) ため、エントリが一致しない場合は描きません。

8 出力例

real data run 3774 event 560 (kpsc entry 560, helix entry 560)

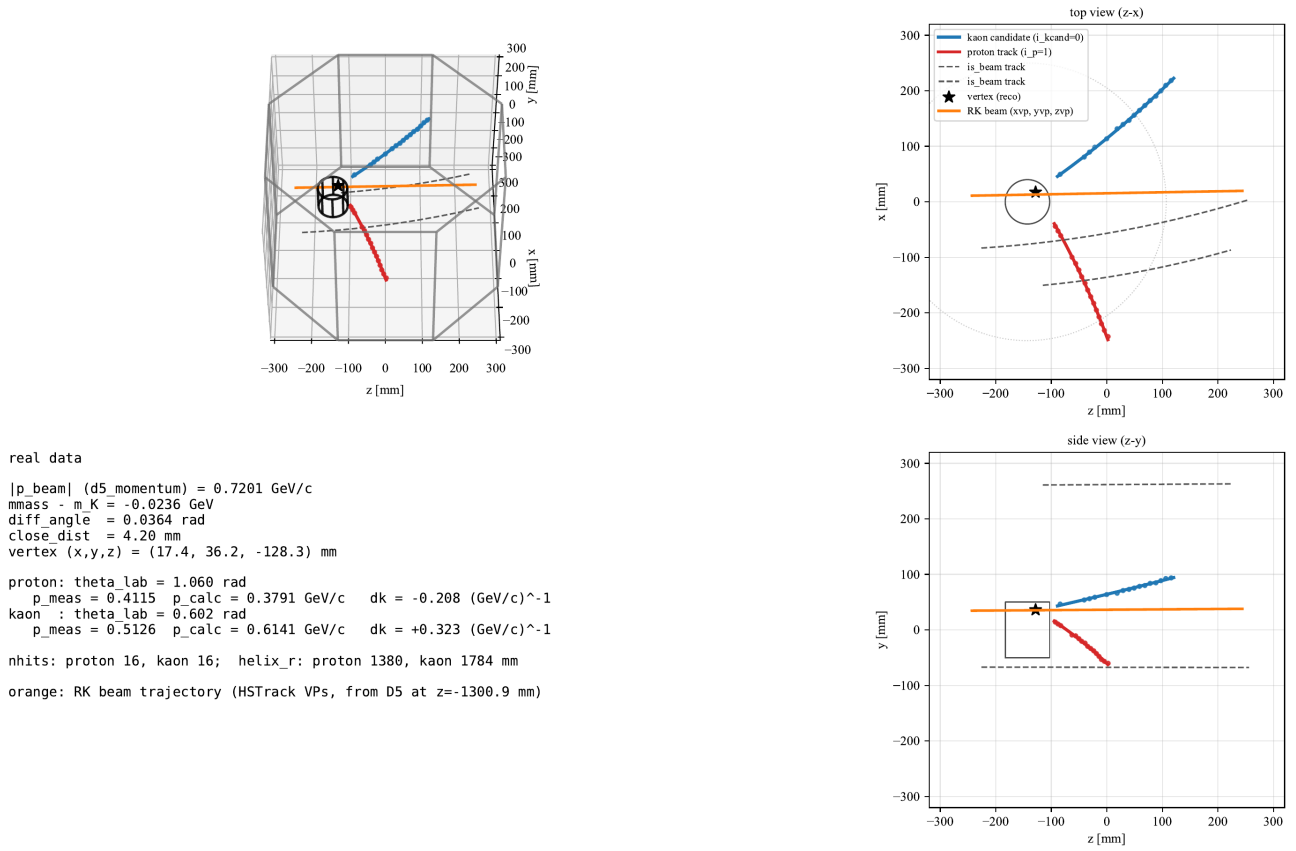


図 1: 実データ (run 3774) の例。

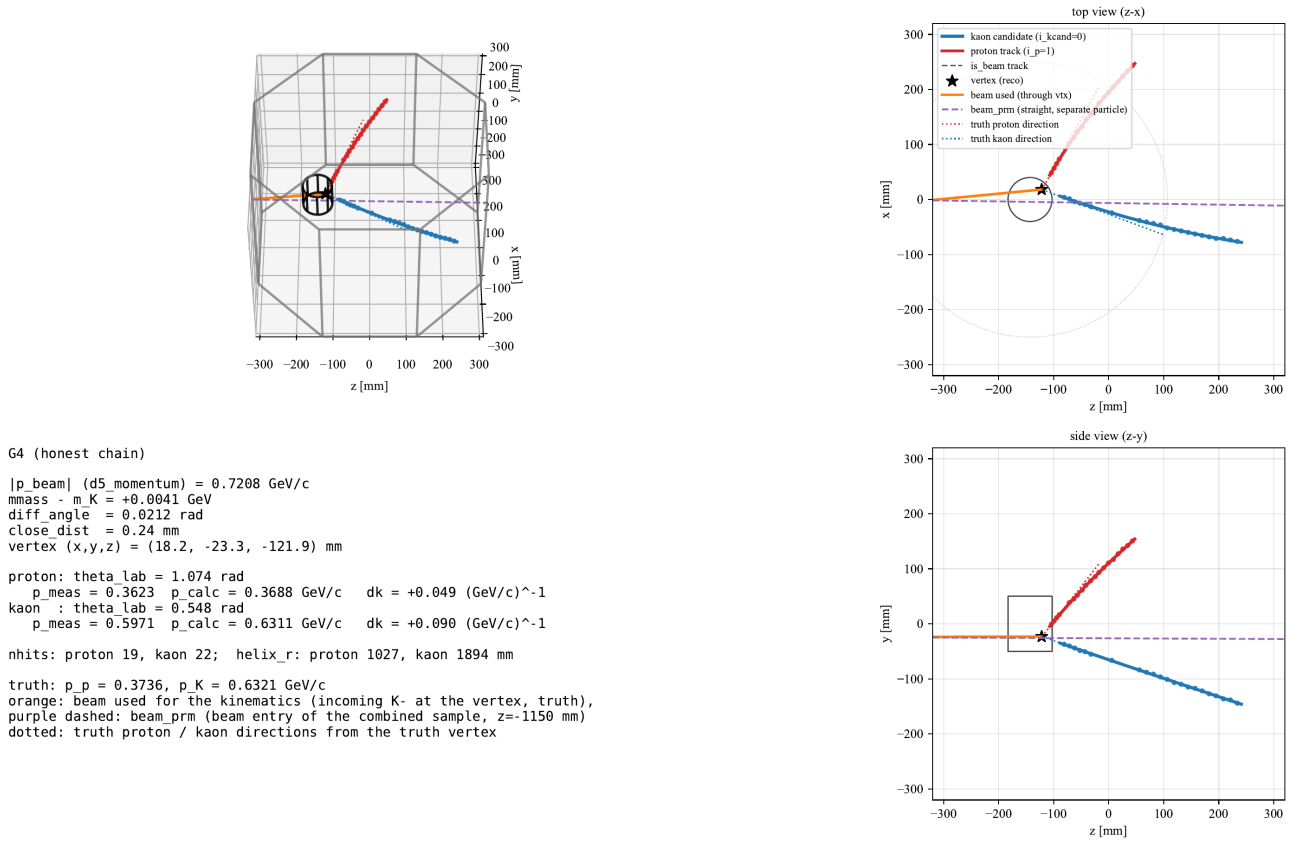


図 2: G4 の例。

9 実行時間とメモリ

- 起動時に helix の event_number (と run_number) と kpsc のスカラー量を全エントリ読みます。実データ 1 ラン (約 82 万エントリ) で、1 イベントの描画に約 30 秒かかります。大部分はこの読み込みです。
- 描画するイベントの helix と xvp, yvp, zvp は、そのエントリだけを読みます。
- --n 12 は、実データ 1 ランで約 40 秒、G4 (5 万エントリ) で約 10 秒です。

10 Python から使う

関数 main にコマンドラインと同じ引数のリストを渡せます。

```

import sys; sys.path.insert(0, "/home/had/sryuta/analyzer/JPARC2025E72/myanalysis/macro")
import kpsc_event_display_rk as ed
ed.main(["--helix", "run03774_TPCHelix.root",
        "--kpsc", "run03774_KpScattering_lh2off.root",
        "--run", "3774", "--event", "560", "--out", "ev.png"])

```